JOTFLOW · GOVERNMENT OF DUBAI MEDIA OFFICE

PostgreSQL Optimization — SQL to Run in pgAdmin

Copy-paste runbook for the `flowaccel` database on the IIS / Windows server. Adds indexes for fast dashboard reads, refreshes planner statistics, and verifies the result.

WHAT YOU'LL DO

1. Connect to the `flowaccel` database and open the Query Tool
2. Create three performance indexes (without locking the table)
3. Refresh planner statistics with `ANALYZE`
4. Verify the indexes are used with `EXPLAIN ANALYZE`
5. Run the diagnostics & routine-maintenance queries

0 Before you start

These statements run against the **same PostgreSQL database your Node backend uses** — the one in `DATABASE_URL` (database name `flowaccel`). Run them in pgAdmin's **Query Tool** on the IIS server.

1. Open **pgAdmin** → expand *Servers* → your PostgreSQL server → *Databases* → `flowaccel`.
2. Right-click `flowaccel` → **Query Tool**.
3. Paste one block at a time and press **F5** (or the ▶ Execute button) to run it.

> **PRODUCTION** This is the live GDMO database. Prefer a low-traffic window. Every statement below is **safe and reversible** — adding an index changes no data, and it can be dropped again (see Step 5). Nothing here deletes or modifies rows.

> **CODE VS SQL** Two of the four optimizations were already made *in the application code* and need no SQL: the dashboard read no longer ships the giant `raw_data` column, and the poller now enriches submissions in parallel. This document covers only the `SQL` part you run in pgAdmin.

1 Create the performance indexes

The dashboard read filters by `form_id` and always sorts by `submission_date` newest-first. The notifications read filters by `user_email` and sorts by `created_at`. These three indexes let PostgreSQL satisfy each filter *and* its sort directly from the index instead of sorting matched rows in memory.

> **RUN ONE AT A TIME** Each uses `CREATE INDEX CONCURRENTLY`, which builds the index **without locking out the poller's writes**. CONCURRENTLY cannot run inside a transaction, so run **each statement on its own** (select just that block, then F5) — do not run all three in a single execution. `IF NOT EXISTS` makes re-running safe.

1a · Submissions — filter by form, sorted by date

```
-- Matches: WHERE form_id = ANY(...) ORDER BY submission_date DESC
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_jf_submissions_form_date
  ON jf_submissions (form_id, submission_date DESC);
```

1b · Submissions — filter by status, sorted by date

```
-- Matches: WHERE status = '...' ORDER BY submission_date DESC
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_jf_submissions_status_date
  ON jf_submissions (status, submission_date DESC);
```

1c · Notifications — filter by user, sorted by date

```
-- Matches: WHERE user_email = '...' ORDER BY created_at DESC LIMIT n
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_notifications_email_created
  ON notifications (user_email, created_at DESC);
```

> **NOTE** These are identical to the indexes now in `backend/db/schema.sql`. If you instead run `npm run migrate` on the server, you don't need to run these by hand — but the migrate path uses a brief table lock, whereas the `CONCURRENTLY` form above does not.

2 Refresh planner statistics

After creating indexes, tell PostgreSQL to recompute table statistics so the query planner knows the new indexes exist and chooses them. This is fast and read-only.

```
ANALYZE jf_submissions;
ANALYZE notifications;
```

Safe to run together. Takes seconds.

3 Verify the indexes are actually used

Run the real dashboard query through `EXPLAIN ANALYZE`. Replace the two form IDs with real ones from your data (grab them from the query in the box below).

Get a couple of real form IDs first

```
SELECT DISTINCT form_id FROM jf_submissions LIMIT 5;
```

Then run the plan

```
EXPLAIN ANALYZE
SELECT * FROM jf_submissions
WHERE form_id = ANY(ARRAY['PASTE_FORM_ID_1', 'PASTE_FORM_ID_2'])
ORDER BY submission_date DESC
LIMIT 200;
```

How to read the result

What you see in the plan	Meaning
`Index Scan using idx_jf_submissions_form_date` (or a `Bitmap` / `Merge` scan over it)	**Working.** The index is being used.
`Seq Scan on jf_submissions` + a large `Sort` node with high `actual time`	Index not used yet — re-run Step 2 (`ANALYZE`). If it persists, the table may simply be small (see below).
Total runtime already < a few ms with a `Seq Scan`	**Fine.** The table is small enough that scanning is faster than an index — PostgreSQL is choosing correctly. No action needed.

> **HONEST NOTE** Indexes only pay off once a table is reasonably large (roughly 10k+ rows). On a small table the planner may ignore them, and that is the correct, fast choice. The bigger speed win for the dashboard was the application-code change that stopped sending the `raw_data` column.

4 Diagnostics — know your data

4a · Table size, index size, and row count

```
SELECT
  pg_size_pretty(pg_total_relation_size('jf_submissions')) AS total_size,
  pg_size_pretty(pg_relation_size('jf_submissions'))       AS table_size,
  pg_size_pretty(pg_indexes_size('jf_submissions'))        AS indexes_size,
  (SELECT count(*) FROM jf_submissions)                    AS row_count;
```

Tells you whether indexes are even worth it (see Step 3 note) and how big `raw_data` made the table.

4b · List every index on the submissions table

```
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'jf_submissions'
ORDER BY indexname;
```

4c · Find UNUSED indexes (don't pay to maintain dead weight)

```
SELECT relname AS table_name,
       indexrelname AS index_name,
       idx_scan AS times_used,
       pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE relname IN ('jf_submissions', 'notifications', 'jf_approval_history')
ORDER BY idx_scan ASC;
```

`times_used = 0` after the app has run for a while means that index is never used and could be dropped. Check again a few days after deploying.

5 Routine maintenance & rollback

5a · Reclaim space & refresh stats (occasional)

PostgreSQL autovacuums automatically, but after a large sync you can run this manually. It does not lock the table for reads/writes.

```
VACUUM (ANALYZE) jf_submissions;
```

5b · Rollback — remove an index if ever needed

Adding an index is fully reversible. Drop concurrently so it never blocks writes:

```
DROP INDEX CONCURRENTLY IF EXISTS idx_jf_submissions_form_date;
DROP INDEX CONCURRENTLY IF EXISTS idx_jf_submissions_status_date;
DROP INDEX CONCURRENTLY IF EXISTS idx_notifications_email_created;
```

5c · Optional — see your slowest queries

If the `pg_stat_statements` extension is enabled, this surfaces the heaviest queries so you know where to look next:

```
SELECT query, calls, mean_exec_time, total_exec_time
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
```

If it errors with "relation does not exist", the extension isn't installed — that's optional and can be skipped.

> **DONE** After Steps 1–3 the dashboard's submission and notification reads are index-backed. Combined with the code-side payload trim, the dashboard fetches the same rows with a much smaller response.